Prof Elective 5 — Advanced Web Development: Back End

Worksheet 4.1 — Laravel Query Builder vs. Eloquent ORM

Guided Coding Activity | Self-paced / Asynchronous Learning

Objective

Compare Laravel Query Builder and Eloquent ORM by implementing equivalent book-retrieval operations in the existing Laravel Books application. Inspect results, examine SQL generated by Eloquent, and explain how query behavior should be evaluated.

Expected Output

A working comparison page that displays books retrieved using both Query Builder and Eloquent, supports title searching and low-stock filtering, sorts results by price, and includes screenshots, code evidence, and a short comparison report.

Before You Start

Use the existing Laravel project from Worksheet 2.4 — CRUD in Laravel: Books. This activity assumes that the books table and app/Models/Book.php already exist. Do not run migrate:fresh; it deletes existing tables and records.

A. Check Your Existing Project

1. Open your existing Laravel project in Visual Studio Code.
2. Open phpMyAdmin and select laravel_project_db.
3. Open the books table and confirm it contains id, title, description, country_id, stock, amount, photo, created_at, and updated_at.
4. Confirm that the table contains book records. If you followed Worksheet 2.4, it should contain sample records created by BookSeeder.
5. Open app/Models/Book.php and confirm that the Book model exists.

Checkpoint 1: Do not continue until the Laravel project connects to the database and books contains records.

B. Create the Query Comparison Page

Create a separate page so your existing Books CRUD page remains unchanged.

1. Create a controller

Open the terminal in your Laravel project and run:

```
php artisan make:controller BookQueryController
```

Open app/Http/Controllers/BookQueryController.php and replace its contents with:

```php
<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;
use Illuminate\Support\Facades\DB;
use App\Models\Book;

class BookQueryController extends Controller
{
    public function index(Request $request)
    {
        $search = $request->input('search', '');
        $lowStock = $request->boolean('low_stock');

        // Laravel Query Builder
        $queryBuilderBooks = DB::table('books')
            ->when($search, function ($query, $search) {
                $query->where('title', 'like', '%' . $search . '%');
            })
            ->when($lowStock, function ($query) {
                $query->where('stock', '<', 10);
            })
            ->orderBy('amount')
            ->limit(20)
            ->get();

        // Eloquent ORM
        $eloquentBooks = Book::query()
            ->when($search, function ($query, $search) {
                $query->where('title', 'like', '%' . $search . '%');
            })
            ->when($lowStock, function ($query) {
                $query->where('stock', '<', 10);
            })
            ->orderBy('amount')
            ->limit(20)
            ->get();

        return view('pages.book-query-comparison', compact(
            'search', 'lowStock',
            'queryBuilderBooks', 'eloquentBooks'
        ));
    }
}
```

Both query chains intentionally use the same conditions, sort order, and limit so the comparison is fair. The low-stock threshold is 10 for this activity.

2. Create the Blade view

Create resources/views/pages/book-query-comparison.blade.php and insert:

```blade
@extends('layouts.app')

@section('content')
<x-page-header pagetitle="Query Builder vs. Eloquent" class="bg-info"/>

<div class="container py-4">
    <form method="GET" action="{{ url('/book-query-comparison') }}"
          class="row g-2 mb-4">
        <div class="col-md-6">
            <input type="text" name="search" class="form-control"
                   placeholder="Search book title..." value="{{ $search }}">
```

```
        </div>
        <div class="col-md-3">
            <label class="form-check mt-2">
                <input class="form-check-input" type="checkbox"
                       name="low_stock" value="1"
                       {{ $lowStock ? 'checked' : '' }}>
                <span class="form-check-label">Stock below 10</span>
            </label>
        </div>
        <div class="col-md-3">
            <button type="submit" class="btn btn-primary">Search</button>
            <a href="{{ url('/book-query-comparison') }}"
               class="btn btn-secondary">Reset</a>
        </div>
    </form>

    <div class="row">
        <div class="col-lg-6 mb-4">
            <h4>Laravel Query Builder</h4>
            <p>Records: {{ $queryBuilderBooks->count() }}</p>
            @forelse($queryBuilderBooks as $book)
                <div class="card mb-2"><div class="card-body">
                    <h5>{{ $book->title }}</h5>
                    <p class="mb-0">Stock: {{ $book->stock }}</p>
                    <p class="mb-0">Price: ₱{{ number_format($book->amount, 2) }}</p>
                </div></div>
            @empty
                <p>No books found.</p>
            @endforelse
        </div>

        <div class="col-lg-6 mb-4">
            <h4>Eloquent ORM</h4>
            <p>Records: {{ $eloquentBooks->count() }}</p>
            @forelse($eloquentBooks as $book)
                <div class="card mb-2"><div class="card-body">
                    <h5>{{ $book->title }}</h5>
                    <p class="mb-0">Stock: {{ $book->stock }}</p>
                    <p class="mb-0">Price: ₱{{ number_format($book->amount, 2) }}</p>
                </div></div>
            @empty
                <p>No books found.</p>
            @endforelse
        </div>
    </div>
</div>
@endsection
```

3. Add the route

Open routes/web.php. Add this import near your other controller imports:

```
use App\Http\Controllers\BookQueryController;
```

Add this route:

```
Route::get('/book-query-comparison', [BookQueryController::class, 'index']);
```

4. Run and open the page

Start the development server:

```
php artisan serve
```

Open the local address shown in the terminal and append /book-query-comparison. For example:

```
http://127.0.0.1:8000/book-query-comparison
```

- The page should show a title-search field, a Stock below 10 checkbox, Search, and Reset.
- Two columns should appear: Laravel Query Builder and Eloquent ORM.
- Both columns should show matching records, ordered by price from lowest to highest, with at most 20 records per column.

Take Screenshot 1 showing the default comparison page.

C. Test the Queries

Test	Action	Expected result
1. All books	Leave search blank and checkbox unchecked; click Search.	Both columns show the same first 20 books, ordered by price ascending.
2. Search	Enter a word found in a book title; click Search.	Both columns show titles containing that word.
3. Low stock	Clear search, check Stock below 10, and click Search.	Both columns show only books with stock below 10.
4. Combined filter	Enter a title keyword and check Stock below 10.	Both columns show books matching both conditions.
5. Reset	Click Reset.	Filters clear and the default list returns.

Take Screenshot 2 showing a title search and Screenshot 3 showing the low-stock filter. Include both columns in each screenshot.

D. Compare the Two Approaches

6. Did both approaches return the same records in all five tests? If not, identify the difference and investigate its cause.
7. Which parts of the code are similar, and which parts differ?
8. Why does the Query Builder example not require a Book model, while the Eloquent example does?
9. Does Book::all() automatically make a query faster than DB::table('books')->get()? Explain.
10. Give one situation where you would prefer Eloquent and one situation where Query Builder may be suitable.

Answer: _____
_____

Answer: _____
_____

Answer: _____
_____

Answer: _____
_____

Answer: _____
_____

E. Inspect SQL Generated by Eloquent

Eloquent ultimately sends SQL queries to the database. Inspect the SQL template and bindings instead of assuming how it works.

1. Temporarily inspect the query

In BookQueryController.php, temporarily replace the Eloquent query section with this code:

```
$eloquentQuery = Book::query()
    ->when($search, function ($query, $search) {
        $query->where('title', 'like', '%' . $search . '%');
    })
    ->when($lowStock, function ($query) {
        $query->where('stock', '<', 10);
    })
    ->orderBy('amount')
    ->limit(20);

dd($eloquentQuery->toSql(), $eloquentQuery->getBindings());
```

Reload the comparison page. It will display the SQL template and bindings. Take Screenshot 4. Then remove dd(...) and restore the original Eloquent query so the page works normally.

Important: dd() is only for local learning. Do not leave it in a deployed application.

SQL template: _____
_____

What the bindings represent:
_____
_____

F. Final Challenge — Add a Custom Stock Threshold

Modify the page so students can enter a stock threshold instead of always using 10.

11. Add a numeric input named stock_limit to the search form.
12. Read and validate the input in the controller as an integer within a reasonable range, such as 0 to 1000.
13. Use the same validated threshold in both the Query Builder and Eloquent queries.
14. Display the selected threshold on the page.
15. Test at least two thresholds and verify that both columns return the same records.

Do not concatenate user input into raw SQL. Validate the threshold and pass it through the query methods.

Take Screenshot 5 showing the completed challenge.

G. Presentation on Submission Requirements

- Default comparison page.
- Title-search results in both columns.
- Low-stock results in both columns.
- SQL template and bindings from Eloquent.
- Final challenge with a custom stock threshold.
- Copy of BookQueryController.php and book-query-comparison.blade.php.
- Answers to Part D and the explanation of SQL and bindings in Part E.

H. Score Sheet

Criteria	Description	Points
Page and route	Controller, view, and route work without errors.	5
Query Builder	Filters and sorting work correctly.	10
Eloquent ORM	Equivalent query returns matching records.	10
SQL inspection	Correctly identifies SQL template and bindings.	10
Final challenge	Validated custom threshold works in both approaches.	10
Analysis and reflection	Answers explain results and justify conclusions, during the presentation.	10
Timeliness	Presented the activity before due date	5
TOTAL		60

I. Troubleshooting Guide

Problem	What to check
Class Book not found	Confirm app/Models/Book.php exists and import App\Models\Book.
View not found	Check resources/views/pages/book-query-comparison.blade.php and view name pages.book-query-comparison.
Route not found	Check routes/web.php and run php artisan route:list.
Database connection error	Check DB settings in .env. In development, php artisan config:clear may help if config is cached.
Both columns empty	Confirm books contains records and filters match existing data.
Results differ	Check both query chains use the same conditions, ordering, and limit.
Page stops at dd()	Remove the temporary dd() statement after inspecting SQL and bindings.

References

- Laravel Documentation — Query Builder: https://laravel.com/docs/12.x/queries
- Laravel Documentation — Eloquent ORM: https://laravel.com/docs/12.x/eloquent
- Laravel Documentation — Database: https://laravel.com/docs/12.x/database
- MySQL Reference Manual — SELECT: https://dev.mysql.com/doc/refman/8.4/en/select.html

Use documentation matching the Laravel version installed in your project. This worksheet uses the existing Books schema from Worksheet 2.4 and requires no additional packages.

If something is not working, figure it out — and document how you solved it.